Enterprise Architecture Standards — Knowledge Base

kb-L1-enterprise-architecture v1.0.0

This KB defines the enterprise-level architecture standards for the loan application platform. All design and construction agents MUST ground their decisions in these standards.

EA1: Technology Stack

Layer	Technology	Version	Purpose	Status
Mobile	Kotlin	2.0	Primary language	Mandatory
Mobile	Jetpack Compose	1.7	UI framework	Mandatory
Mobile	Material Design 3	Latest	Design system	Mandatory
Mobile	AndroidX	Latest	Core libraries	Mandatory
Mobile	CameraX	Latest	Document capture & selfie	Mandatory
Mobile	Room	2.6	Local persistence	Mandatory
Mobile	WorkManager	2.9	Background processing	Mandatory
Mobile	OkHttp	4.12	HTTP client	Mandatory
Mobile	Retrofit	2.11	REST client	Mandatory
Mobile	Hilt	2.51	Dependency injection	Mandatory
Mobile	Navigation Compose	2.8	Screen routing	Mandatory
Backend	C#	12	Primary language	Mandatory
Backend	.NET 8 Web API	8	API framework	Mandatory
Backend	Entity Framework Core	8	ORM	Mandatory
Backend	FluentValidation	11	Request validation	Mandatory
Backend	Serilog	4	Structured logging	Mandatory
Backend	Polly	8	Resilience & fault handling	Mandatory
Backend	MediatR	12	CQRS / mediator pattern	Mandatory
Backend	AutoMapper	13	Object mapping	Approved
Backend	Swashbuckle	6	OpenAPI / Swagger	Mandatory
Database	SQL Server	2022	Primary database	Mandatory
Database	Redis	7.2	Caching	Mandatory
Infrastructure	Azure Kubernetes Service	Latest	Container orchestration	Mandatory
Infrastructure	Azure API Management	Latest	API gateway	Mandatory
Infrastructure	Azure Key Vault	Latest	Secrets management	Mandatory
Infrastructure	Azure Monitor	Latest	Observability	Mandatory
Infrastructure	Azure DevOps	Latest	CI/CD pipelines	Mandatory
Testing	xUnit	2.8	Unit testing (.NET)	Mandatory
Testing	NSubstitute	5	Mocking (.NET)	Mandatory

Layer	Technology	Version	Purpose	Status
Testing	Testcontainers .NET	Latest	Integration testing	Mandatory
Testing	WireMock.Net	Latest	HTTP mocking	Mandatory
Testing	Espresso	Latest	Android UI testing	Mandatory
Testing	Compose UI Test	Latest	Compose testing	Mandatory
Mobile	Firebase Crashlytics	Latest	Crash reporting	Mandatory
Mobile	Firebase Performance	Latest	Performance monitoring	Mandatory
Mobile	Coil	2.6	Image loading & caching	Mandatory
Infrastructure	Azure Service Bus	Latest	Async messaging	Mandatory
Infrastructure	Azure App Configuration	Latest	Feature flags & config	Mandatory
Security	SonarQube	Latest	SAST analysis	Mandatory
Security	OWASP ZAP	Latest	DAST analysis	Mandatory
Security	Snyk	Latest	Dependency vulnerability scanning	Mandatory

MANDATORY RULE: Agents MUST NOT introduce technologies not on this list. If a requirement needs unlisted tech, output `UNLISTED_TECHNOLOGY_REQUIRED:[name]` for ARB review.

EA2: Architecture Patterns

Backend: Clean Architecture

Four layers with strict dependency direction (inward only):

1. **Domain** — Entities, value objects, domain events, repository interfaces. Zero external dependencies.
2. **Application** — Use cases/handlers (MediatR), DTOs, validators (FluentValidation). Depends only on Domain.
3. **Infrastructure** — EF Core DbContext, repository implementations, external service clients. Implements interfaces from Domain/Application.
4. **Presentation** — Controllers, middleware, filters. Entry point; depends on Application.

Mobile: MVVM with Unidirectional Data Flow

- ViewModel (Hilt-injected) → StateFlow → Compose UI.
- Repository pattern for data access.
- Room for local persistence.
- WorkManager for background sync.
- Navigation Compose for routing.

Communication

- **Synchronous (REST over HTTPS):** Client-to-server communication.
- **Asynchronous (Azure Service Bus):** Inter-service events (e.g., `application-submitted`, `kyc-completed`, `offer-generated`).
- **Event-driven:** All status changes propagated via events.

State Management

- **Server-side:** State machine for loan application lifecycle: `Draft → Submitted → Verifying → UnderReview → Approved → OfferGenerated → OfferAccepted → Disbursed → Active → Closed`
- **Mobile-side:** SavedStateHandle for process death recovery. Room for persistence.

CQRS (Command Query Responsibility Segregation)

- Commands (write operations) handled via MediatR `IRequestHandler<TCommand, TResult>`.
- Queries (read operations) handled via MediatR `IRequestHandler<TQuery, TResult>`.
- Commands go through validation pipeline (FluentValidation behavior).
- Queries may bypass validation for performance.
- Separate read models for complex queries (e.g., dashboard aggregations).

Domain-Driven Design

- Bounded contexts: Auth, Products, Applications, KYC, Offers, Loans, Notifications.
- Each bounded context owns its schema, entities, and events.
- Cross-context communication via domain events (Azure Service Bus).
- Shared kernel: common value objects (Money, DateRange, Address) in a shared library.
- Anti-corruption layer for third-party integrations (KYC provider, credit bureau).

Solution Structure

Backend (.NET):

```
src/
  LoanApp.Domain/           # Entities, value objects, interfaces, domain events
  LoanApp.Application/      # Use cases, DTOs, validators, MediatR handlers
  LoanApp.Infrastructure/   # EF Core, external clients, message bus
  LoanApp.Api/              # Controllers, middleware, startup
  LoanApp.Shared/           # Shared kernel (Money, DateRange, etc.)
tests/
  LoanApp.Domain.Tests/
  LoanApp.Application.Tests/
  LoanApp.Infrastructure.Tests/
  LoanApp.Api.Tests/        # Integration tests
```

Mobile (Kotlin):

```
app/
  src/main/java/com/loanapp/
    core/           # DI modules, base classes, extensions
    data/           # Repositories, Room DAOs, Retrofit APIs, DTOs
    domain/         # Use cases, models, repository interfaces
    ui/
      auth/         # Login, Register, OTP screens + ViewModels
      products/     # Product list, detail, compare screens + ViewModels
      application/  # Form steps, review screens + ViewModels
      kyc/          # Document capture, selfie, status screens + ViewModels
      offer/        # Offer display, calculator screens + ViewModels
      dashboard/    # Dashboard, payments, history screens + ViewModels
      notifications/ # Notification list, preferences screens + ViewModels
      components/   # Design system (atoms, molecules, organisms)
      theme/        # Colors, typography, shapes, dimensions
    workers/        # WorkManager workers (sync, notifications)
```

Error Handling Pattern

Backend:

- Global exception middleware converts all exceptions to RFC 7807 ProblemDetails.
- Domain exceptions: `BusinessRuleViolationException` -> 422, `EntityNotFoundException` -> 404, `ConflictException` -> 409.
- Validation exceptions: FluentValidation failures -> 400 with field-level errors.
- Infrastructure exceptions: DB timeout -> 503, external service failure -> 502.
- Unhandled exceptions -> 500 with correlation ID (no stack trace in response).

Mobile:

- `Result<T>` sealed class: `Success(data)`, `Error(exception, message)`.
- Repository layer catches all exceptions and returns `Result<T>`.
- ViewModel maps `Result<T>` to `UiState` (Loading, Success, Error).
- Error UI: user-friendly message + retry action. Technical details logged to Crashlytics.
- Network errors: "Connection issue - your progress is saved. Tap to retry."
- Server errors: "Something went wrong. Please try again."
- Validation errors: inline field-level messages.

MANDATORY RULES:

- All backend services MUST follow Clean Architecture layers.
- All mobile screens MUST use ViewModel + StateFlow.
- No direct database access from controllers/screens.
- All inter-service communication MUST go through defined events.
- All backend exceptions MUST be caught by global middleware and returned as ProblemDetails.
- All mobile API calls MUST return Result - no raw exceptions in ViewModel.

EA3: API Standards

- **Specification:** OpenAPI 3.0 required for all APIs.
- **Base URL pattern:** `/api/v{version}/{resource}`
- **Versioning:** URL path versioning (v1, v2). Breaking changes = new version. Non-breaking additions allowed in current version.

HTTP Methods

Method	Usage
GET	Read
POST	Create / action
PUT	Full replace
PATCH	Partial update
DELETE	Soft delete only

Request/Response

- JSON only. camelCase property names. ISO 8601 dates.
- Pagination via offset/limit with Link headers.
- Response envelope:

```
{
  "data": "...",
  "meta": { "page": 1, "pageSize": 20, "totalCount": 100 },
  "errors": []
}
```

Error Format

RFC 7807 Problem Details. Standard error codes:

Code	Meaning
400	Validation error
401	Unauthenticated
403	Forbidden
404	Not found
409	Conflict / idempotency
422	Business rule violation
429	Rate limited
500	Internal server error
503	Service unavailable

Rate Limiting

- Read endpoints: 100 req/min per user.
- Write endpoints: 20 req/min per user.

- 429 response with `Retry-After` header.

Idempotency

- All POST/PUT/PATCH must accept `Idempotency-Key` header.
- Server stores result for 24 hours.
- Duplicate request returns cached response.

HATEOAS

Not required but links to related resources encouraged.

API Naming Conventions

- Resource names: plural nouns (`/products`, `/applications`, `/loans`). Never verbs.
- Actions on resources: POST with action name (`/applications/{id}/submit`, `/offers/{id}/accept`).
- Nested resources for ownership: `/applications/{id}/documents`, `/loans/{id}/payments`.
- Query parameters: camelCase (`pageSize`, `sortBy`, `filterStatus`).

Standard Request Headers

Header	Required	Description
`Authorization`	Yes (except public)	`Bearer {jwt}`
`Content-Type`	Yes (POST/PUT/PATCH)	`application/json`
`Accept`	Yes	`application/json`
`X-Correlation-Id`	Auto-generated	UUID, propagated across services
`Idempotency-Key`	Yes (POST/PUT/PATCH)	UUID, client-generated
`X-Device-Id`	Yes (mobile)	Device fingerprint for session binding
`Accept-Language`	Optional	`en-GB` default

Standard Response Headers

Header	Description
`X-Correlation-Id`	Echo back request correlation ID
`X-RateLimit-Limit`	Max requests per window
`X-RateLimit-Remaining`	Remaining requests
`X-RateLimit-Reset`	Window reset time (Unix epoch)
`Cache-Control`	Caching directive
`ETag`	Resource version for conditional requests

File Upload Standards

- Endpoint pattern: `POST /resources/{id}/documents`
- Content-Type: `multipart/form-data`
- Max file size: 10MB per file, 25MB per request.
- Allowed types: `image/jpeg`, `image/png`, `application/pdf`.
- Virus scanning on upload (before storage).
- Response: 201 with document metadata (id, name, size, mimeType, uploadedAt).

Health Check Endpoints

- `GET /health` — liveness probe. Returns 200 if process is running. No dependency checks.
- `GET /health/ready` — readiness probe. Checks: DB connection, Redis connection, Service Bus connection. Returns 200 if all healthy, 503 if any unhealthy.
- Response body: `{"status": "Healthy", "checks": {"database": "Healthy", "redis": "Healthy", "serviceBus": "Healthy"}}`

MANDATORY RULES:

- All endpoints MUST have OpenAPI spec before implementation.
- All responses MUST use RFC 7807 for errors.
- All write endpoints MUST support idempotency.
- No PII in URL paths or query parameters.
- All APIs MUST use plural noun resource names.
- All requests MUST include X-Correlation-Id (auto-generated if missing).
- All file uploads MUST be virus-scanned before storage.

EA4: Data Architecture

- **Primary database:** SQL Server 2022 with EF Core 8.

Schema Design

One schema per bounded context: `auth`, `products`, `applications`, `kyc`, `offers`, `loans`, `notifications`. Cross-schema access via application layer only — no cross-schema joins.

Naming Conventions

Element	Convention	Example
Tables	PascalCase plural	`Users`, `LoanApplications`
Columns	PascalCase	`FirstName`, `CreatedAt`
Foreign keys	`{ReferencedTable}Id`	`UserId`
Indexes	`IX_{Table}_{Column}`	`IX_Users_Email`

Required Columns (Every Table)

Column	Type	Constraint
Id	UNIQUEIDENTIFIER	PK, default `NEWSEQUENTIALID()`
CreatedAt	DATETIMEOFFSET	NOT NULL
CreatedBy	NVARCHAR(100)	NOT NULL
UpdatedAt	DATETIMEOFFSET	NULL
UpdatedBy	NVARCHAR(100)	NULL
IsDeleted	BIT	default 0
Version	ROWVERSION	Optimistic concurrency

Soft Delete

All deletes set `IsDeleted=1`. Hard delete prohibited except for GDPR right-to-erasure (requires audit log entry).

Encryption

PII columns encrypted at application level (AES-256) before storage: `NationalId`, `DateOfBirth`, `PhoneNumber`, `Email`, `Address`, `IncomeDetails`, `BankStatements`. Encryption key managed via Azure Key Vault.

Migrations

- EF Core code-first migrations.
- Naming: `V{N}__{Description}.cs`.
- All migrations must be idempotent.
- Rollback script required for every migration.

Caching (Redis 7.2)

Data	TTL	Pattern
Product catalog	1 hour	Cache-aside
Session data	15 minutes (matches JWT)	Cache-aside

Cache invalidation on write.

Data Classification

Classification	Examples
Public	Product catalog, rates
Internal	Application status, system config
Confidential	Name, email, employment
Restricted	National ID, income, bank statements, biometric data

Connection Pooling

- EF Core default connection pool: min 0, max 100 per service instance.
- Connection timeout: 30 seconds.
- Command timeout: 30 seconds (60 seconds for reporting queries).
- Connection string in Azure Key Vault, injected via Azure App Configuration.

Query Performance Guidelines

- No `SELECT *` — always specify columns needed (use `.Select()` projections).
- Indexes required for: all foreign keys, all columns used in WHERE clauses, all columns used in ORDER BY.
- Composite indexes for common query patterns (e.g., `IX_Applications_UserId_Status`).
- Use `.AsNoTracking()` for read-only queries.
- Pagination required for all list endpoints (max 100 items per page).
- N+1 query detection: use `.Include()` for related entities, never lazy loading.
- Complex aggregations: use raw SQL or stored procedures via `FromSqlRaw()`.

Transaction Isolation

Operation Type	Isolation Level	Rationale
Standard reads	Read Committed	Default, sufficient for most queries
Financial operations	Serializable	Offer acceptance, payment recording — must prevent phantom reads
Reporting queries	Read Committed Snapshot (RCSI)	Non-blocking reads for dashboards

Backup Strategy

- Full backup: daily at 02:00 UTC.
- Differential backup: every 6 hours.
- Transaction log backup: every 15 minutes (supports RPO < 15min).
- Backup retention: 30 days.
- Geo-redundant backup storage (Azure paired region).
- Restore tested monthly.

EF Core Configuration Conventions

- One `IEntityTypeConfiguration<T>` class per entity.
- Configuration classes in `Infrastructure/Persistence/Configurations/` folder.
- Global query filter for soft delete: `.HasQueryFilter(e => !e.IsDeleted).`
- Value converters for encrypted fields.
- Owned types for value objects (Money, Address).

Data Migration Strategy (Zero-Downtime)

- All schema changes must be backward-compatible with the previous application version (supports rolling deployments).
- Additive changes only in a single release: add columns (nullable or with default), add tables, add indexes. Never remove or rename in the same release.
- Destructive changes (remove column, rename column, change type) require a 2-release process:
  - Release 1: add new column, deploy code that writes to both old and new, backfill existing data.
  - Release 2: remove old column, deploy code that only uses new.
- Migration execution: EF Core migrations run as Kubernetes init container before app pods start. If migration fails, deployment is rolled back.
- Rollback: every migration must have a corresponding reverse migration. Tested in staging before production.
- Large table migrations: for tables with >1M rows, use online schema change tools (e.g., pt-online-schema-change equivalent for SQL Server) to avoid table locks.
- Data backfill: for new non-nullable columns on existing tables, provide a backfill script that runs as a separate job (not in the migration itself).

MANDATORY RULES:

- All PII MUST be encrypted at rest.
- All tables MUST have audit columns.
- No cross-schema joins.
- Soft delete only (except GDPR erasure).
- All queries MUST use parameterized queries (EF Core handles this).
- No SELECT * — always project specific columns.

- All list endpoints MUST be paginated (max 100 per page).
- Financial operations MUST use Serializable isolation.

EA5: Security Architecture

Authentication

- OAuth 2.0 Authorization Code flow with PKCE (mobile).
- JWT access tokens: RS256, 15-minute expiry.
- Refresh tokens: opaque, 24-hour expiry, rotated on use, bound to device fingerprint.
- Biometric authentication via AndroidX BiometricPrompt with credential stored in Android Keystore.

Authorization

- Role-based: `Customer`, `Agent`, `Admin`, `System`.
- Claims-based for fine-grained permissions.
- API Gateway validates JWT signature and expiry.
- Backend validates claims and roles.

Password Policy

- Minimum 12 characters, mixed case + number + special character.
- BCrypt hashing (work factor 12).
- Account lockout after 5 failed attempts (30-minute progressive lockout).
- Password history: last 5 passwords blocked.

OTP

- 6-digit numeric, 90-second expiry, single-use.
- Rate limited: 3 per 10 minutes per phone/email.
- Delivered via SMS gateway or email.
- SHA-256 hashed in storage.

Transport Security

- TLS 1.2+ mandatory.
- Certificate pinning on mobile (backup pin required).
- HSTS headers on all responses.
- No mixed content.

API Security

- API key for service-to-service.
- JWT for user-to-service.
- CORS restricted to known origins.
- Headers: `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`.

PII Handling

- No PII in logs (Serilog destructuring exclusions).
- No PII in URLs.
- No PII in error messages.
- PII masked in non-production environments.
- Data minimization: collect only what's needed for the loan decision.

Session Management (Multi-Device)

- Maximum concurrent sessions per user: 3 (configurable).
- When a 4th device logs in: oldest session is invalidated (FIFO). User notified on the invalidated device.
- Each session bound to device fingerprint (device ID + OS version + app version).
- Login from a new (unrecognised) device triggers: push notification to existing devices ("New login from "), optional: require SCA on new device.
- Session list: user can view active sessions in settings (device name, last active, location). Can revoke any session.
- Suspicious activity: if login from a new device in a different country within 1 hour of last activity, flag for fraud review and require SCA.

Secrets Management

- Azure Key Vault for all secrets: DB connection strings, API keys, encryption keys, JWT signing keys.
- No secrets in code, config files, or environment variables.
- Rotation policy: JWT signing keys every 90 days, API keys every 180 days, DB passwords every 90 days.

Input Validation & Sanitization

- All input validated at API boundary (FluentValidation).
- String inputs: max length enforced, HTML stripped, SQL injection prevented (parameterized queries via EF Core).
- Numeric inputs: range validation (e.g., loan amount min/max, income > 0).
- File inputs: type validation (magic bytes, not just extension), size validation, virus scan.
- Mobile: client-side validation for UX, server-side validation as source of truth.

OWASP Top 10 Mapping

OWASP Risk	Mitigation
A01: Broken Access Control	Role + claims-based auth, API Gateway JWT validation
A02: Cryptographic Failures	AES-256 at rest, TLS 1.2+ in transit, Key Vault for keys
A03: Injection	Parameterized queries (EF Core), input validation
A04: Insecure Design	Threat modeling per feature, security stories in backlog
A05: Security Misconfiguration	Hardened container images, no default credentials, security headers
A06: Vulnerable Components	Snyk scanning in CI, automated dependency updates
A07: Auth Failures	OAuth 2.0 + PKCE, MFA/SCA, account lockout, JWT rotation
A08: Data Integrity Failures	Signed JWTs (RS256), integrity checks on file uploads
A09: Logging Failures	Structured logging, audit trail, no PII in logs

OWASP Risk	Mitigation
A10: SSRF	No user-controlled URLs in server-side requests, allowlist for external calls

Mobile App Security

- Root/jailbreak detection: warn user, block biometric auth on rooted devices.
- Screenshot prevention on sensitive screens (FLAG_SECURE): login, OTP, income details, offer acceptance.
- App attestation: Google Play Integrity API to verify genuine app + device.
- Certificate pinning with backup pins (rotate without app update).
- No sensitive data in app backups (`android:allowBackup="false"` for sensitive data).
- Obfuscation: R8/ProGuard with optimized rules for all release builds.

Dependency Vulnerability Management

- Snyk scan on every PR (block merge if critical/high vulnerabilities).
- Weekly automated dependency update PRs (Renovate).
- Quarterly manual review of all dependencies.
- No dependencies with known critical CVEs in production.

MANDATORY RULES:

- All endpoints MUST require authentication (except `/health` and `/products` public listing).
- All PII MUST be encrypted at rest and in transit.
- No secrets in code or config.
- Biometric data MUST NOT leave the device.
- All authentication events MUST be audit logged.
- All input MUST be validated server-side (client validation is UX only).
- Sensitive screens MUST prevent screenshots (FLAG_SECURE).
- All PRs MUST pass Snyk vulnerability scan.

EA6: Infrastructure & Deployment

Container Orchestration

- Azure Kubernetes Service (AKS).
- One namespace per environment: `dev`, `staging`, `prod`.
- Resource limits on all pods.
- Horizontal Pod Autoscaler (HPA) based on CPU/memory.

API Gateway

- Azure API Management: rate limiting, JWT validation, request/response transformation, API versioning, developer portal.

CI/CD

- Azure DevOps Pipelines.

- Trunk-based development with short-lived feature branches.
- Branch naming: `feature/{jira-key}-{description}`, `aava/{agent-generated-description}`.
- PR required for main. Minimum 1 approval. All CI checks must pass.

Pipeline Stages

`Build → Unit Test → SAST (SonarQube) → Container Build → Integration Test → DAST (OWASP ZAP) → Deploy to Staging → Smoke Test → Deploy to Prod (manual gate) → Post-deploy verification`

Environments

Environment	Trigger	Notes
dev	Auto-deploy on PR merge	—
staging	Auto-deploy on release branch	—
prod	Manual approval gate	—

Environment parity: all environments use same container images, different config via Azure App Configuration.

Database Deployment

- EF Core migrations run as init container in Kubernetes.
- Rollback via reverse migration.
- Blue-green not used for DB (forward-only with backward compatibility).

Mobile Deployment

- Android App Bundle via Google Play Console.
- Rollout: Internal testing → Closed beta → Open beta → Production (staged: 1% → 10% → 50% → 100%).
- Firebase App Distribution for internal builds.

Container Image Standards

- Base image: `mcr.microsoft.com/dotnet/aspnet:8.0-alpine` (minimal attack surface).
- Multi-stage build: build stage uses SDK image, runtime uses aspnet image.
- Image size limit: < 200MB.
- No root user in container (run as non-root).
- Vulnerability scan on every build (Trivy).
- Image signing (Cosign) for supply chain security.

Kubernetes Resource Standards

Resource	CPU Request	CPU Limit	Memory Request	Memory Limit
API service	250m	500m	256Mi	512Mi
Background worker	100m	250m	128Mi	256Mi
Redis	100m	250m	128Mi	256Mi

- Pod Disruption Budget: `minAvailable: 1` for all services.
- Replica count: min 2, max 10 (HPA).
- Liveness probe: `/health` every 10s, failure threshold 3.

- Readiness probe: `/health/ready` every 5s, failure threshold 3.

Network Policies

- Default deny all ingress.
- Allow: API Gateway -> API services (port 8080).
- Allow: API services -> SQL Server (port 1433).
- Allow: API services -> Redis (port 6379).
- Allow: API services -> Service Bus (port 5671).
- No direct pod-to-pod communication outside defined policies.

Disaster Recovery

- Active-passive setup across Azure paired regions.
- Database: geo-replication with automatic failover group.
- Redis: geo-replication.
- Service Bus: geo-disaster recovery (paired namespace).
- DNS failover via Azure Traffic Manager.
- DR drill: quarterly (full failover test).

MANDATORY RULES:

- All deployments MUST go through CI/CD pipeline. No manual deployments.
- All containers MUST have resource limits.
- All environments MUST use same container images.
- Production deployments MUST have manual approval gate.
- All containers MUST run as non-root.
- All container images MUST pass vulnerability scan.
- All services MUST have Pod Disruption Budget.

EA7: Non-Functional Requirements

Performance

Metric	Target
API p95 latency	< 500ms
API p99 latency	< 1000ms
App cold start	< 2s
App warm start	< 500ms
Form completion (5-step wizard)	< 5 minutes
Database query p95	< 100ms

Availability

- 99.95% uptime (≤ 22 minutes downtime/month).
- RTO < 1 hour. RPO < 15 minutes.
- Health check endpoints: `/health` (liveness), `/health/ready` (readiness).

Scalability

- 10,000 concurrent users.
- 1,000 loan applications per hour.
- Horizontal scaling via AKS HPA.
- Database read replicas for reporting.

Resilience

- Circuit breaker on all external service calls (Polly).
- Retry with exponential backoff: 1s, 2s, 4s, max 3 retries.
- Bulkhead isolation per service.
- Graceful degradation: if KYC service is down, allow application save but block submission.
- Timeout: 30s for API calls, 60s for file uploads.

Mobile Resilience

- Offline form saving (Room DB).
- Background sync (WorkManager).
- Crash recovery (SavedStateHandle + Room).
- Network error detection (ConnectivityManager).
- Exponential backoff on retry.

Accessibility

- WCAG 2.1 AA compliance.
- TalkBack support on all screens.
- Minimum 48dp touch targets.
- Dynamic text sizing (respect system font scale).
- High contrast support.
- Logical focus order.
- `contentDescription` on all interactive elements.

MANDATORY RULES:

- All APIs MUST meet p95 < 500ms.
- All external calls MUST have circuit breaker + retry.
- All mobile forms MUST support offline save.
- All screens MUST be WCAG 2.1 AA compliant.

EA8: Integration Patterns

Synchronous (REST)

- Flow: Mobile app → API Gateway → Backend services.
- Used for: user actions (login, submit form, accept offer), data retrieval (product list, loan status).
- Timeout: 30s. Retry: client-side with exponential backoff.

Asynchronous (Azure Service Bus)

Topics and events:

Topic	Events
application-events	`application.submitted`, `application.approved`, `application.rejected`
kyc-events	`kyc.started`, `kyc.completed`, `kyc.failed`
offer-events	`offer.generated`, `offer.accepted`, `offer.declined`, `offer.expired`
loan-events	`loan.created`, `loan.payment.due`, `loan.payment.received`
notification-events	`notification.send`

- Dead letter queue for failed messages. Max delivery count: 5.

Canonical Event Schema

```
{
  "eventId": "uuid",
  "eventType": "domain.action",
  "source": "service-name",
  "timestamp": "ISO8601",
  "correlationId": "uuid",
  "data": {},
  "metadata": {
    "version": "1.0",
    "userId": "uuid"
  }
}
```

Third-Party Integrations

Integration	Protocol	Pattern
KYC provider	REST	Async callback
Credit bureau	REST	Synchronous
SMS gateway	REST	Fire-and-forget with retry
FCM (push notifications)	REST	Fire-and-forget with retry
UK Address Lookup API	REST	Synchronous, cached

Integration Resilience

All third-party calls wrapped in circuit breaker. Fallback behavior per integration:

Integration	Fallback
KYC	Queue for retry
Credit bureau	Block submission until available
SMS	Queue and retry
FCM	Queue and retry
Address lookup	Allow manual entry

Webhook Standards (Inbound Callbacks)

- Used for: KYC provider verification results, payment processor status updates, Open Banking consent notifications.
- Endpoint pattern: `POST /api/v1/webhooks/{provider}` (e.g., `/api/v1/webhooks/onfido`, `/api/v1/webhooks/stripe`).
- Authentication: HMAC signature verification. Provider signs payload with shared secret. Server verifies signature before processing. Reject unsigned or invalid requests with 401.
- Idempotency: webhooks may be delivered multiple times. Use event ID for deduplication. Process each event ID only once.
- Response: return 200 immediately (within 5 seconds). Process webhook payload asynchronously via message queue. If processing fails, the event is retried from the queue (not from the provider).
- Retry from provider: if server returns non-2xx, provider will retry (typically 3-5 times with exponential backoff). Server must handle retries gracefully (idempotency).
- Logging: log all webhook receipts (provider, event type, event ID, signature valid, processing status). No PII in logs.
- Monitoring: alert if webhook delivery rate drops (provider may be having issues) or if signature validation failure rate spikes (potential attack).

MANDATORY RULES:

- All inter-service communication MUST use defined event topics.
- All events MUST follow canonical schema.
- All third-party integrations MUST have circuit breaker + fallback.
- No direct service-to-service REST calls (use events or API Gateway).

EA9: Observability Standards

Logging

- Serilog with structured JSON output.
- Correlation ID on every log entry (propagated via HTTP header `X-Correlation-Id`).
- NO PII in logs — use Serilog destructuring exclusions for sensitive properties.

Level	Usage
Debug	Dev only
Information	Request/response summary
Warning	Degraded performance, retry
Error	Failed operation
Fatal	Service crash

Log format:

```
{
  "timestamp": "ISO8601",
  "level": "Information",
  "correlationId": "uuid",
  "service": "name",
  "message": "text",
  "properties": {},
  "exception": {}
}
```

Metrics

Azure Monitor + Application Insights. Standard metrics:

Metric	Type
`request_duration_ms`	Histogram
`request_count`	Counter (by endpoint, status)
`active_connections`	Gauge
`error_rate`	Counter
`circuit_breaker_state`	Gauge (per dependency)
`queue_depth`	Gauge (per topic)
`cache_hit_ratio`	Gauge

Tracing

- Distributed tracing via OpenTelemetry.
- Trace context propagated via W3C Trace Context headers.
- Spans for: HTTP requests, database queries, cache operations, message publish/consume, external API calls.

Alerting

Priority	Condition	Action
P1 (page)	Error rate > 5% for 5min, p95 latency > 2s for 5min, service unavailable	Page on-call
P2 (ticket)	Error rate > 1% for 15min, p95 latency > 1s for 15min, circuit breaker open	Create ticket
P3 (dashboard)	Cache hit ratio < 80%, queue depth > 100	Dashboard alert

Dashboards

- **Service health:** Per service — request rate, error rate, latency percentiles, availability.
- **Business metrics:** Applications submitted/hour, approval rate, average processing time.
- **Infrastructure:** CPU, memory, pod count, node health.

Mobile Observability

- Firebase Crashlytics for crash reporting.

- Firebase Performance Monitoring for screen render time, network latency.
- Custom events: `form_step_completed`, `application_submitted`, `offer_viewed`, `offer_accepted`.

MANDATORY RULES:

- All services MUST emit structured logs with correlation ID.
- All services MUST expose `/health` and `/health/ready`.
- All external calls MUST have tracing spans.
- No PII in logs, metrics, or traces.

EA10: Mobile Architecture

Architecture

MVVM + Clean Architecture. Layers:

- **UI:** Compose screens + ViewModels.
- **Domain:** Use cases, models.
- **Data:** Repositories, Room DAOs, Retrofit APIs.
- Dependency injection via Hilt.

Navigation

- Navigation Compose with type-safe arguments.
- Deep link support for notifications.
- Single Activity architecture.
- NavHost with nested graphs per feature: `auth`, `products`, `application`, `kyc`, `offer`, `dashboard`, `notifications`, `settings`.

State Management

- ViewModel exposes `StateFlow<UiState>`.
- `UiState` is a sealed class: `Loading`, `Success`, `Error`.
- One-time events via `SharedFlow` (navigation, snackbar).
- No LiveData — StateFlow only.

Networking

- Retrofit 2.11 + OkHttp 4.12.
- Single `RetrofitClient` instance (Hilt singleton).
- OkHttp `Authenticator` for automatic token refresh.
- Certificate pinning via OkHttp `CertificatePinner`.
- Request/response logging in debug builds only.

Local Storage

Store	Usage

Store	Usage
Room 2.6	Application drafts, cached products, notification preferences
EncryptedSharedPreferences	Tokens and sensitive config
Android Keystore	Biometric credentials

No plain `SharedPreferences` for sensitive data.

Background Work (WorkManager 2.9)

Task	Type	Policy
Form sync	OneTimeWorkRequest	`ExistingWorkPolicy.REPLACE`
Notification token refresh	PeriodicWorkRequest	24hr interval
Offline queue processing	OneTimeWorkRequest	Constraint: `NetworkType.CONNECTED`

Image Handling

- CameraX for document capture and selfie.
- Coil for image loading and caching.
- Image compression before upload (max 2MB).
- EXIF data stripped before upload (privacy).

Build Configuration

- Build variants: `debug`, `staging`, `release`.
- Product flavors: none (single app).
- ProGuard/R8 for release builds.
- `BuildConfig` fields for API base URL per environment.

MANDATORY RULES:

- All screens MUST use ViewModel + StateFlow.
- No direct API calls from Compose screens.
- All sensitive data MUST use EncryptedSharedPreferences or Keystore.
- All images MUST strip EXIF before upload.
- All network calls MUST go through single RetrofitClient.

EA11: Design System Standards

Framework

Jetpack Compose with Material Design 3.

Atomic Design Hierarchy

Level	Components
Atoms	Button, TextField, Icon, Text, Divider, Spacer
Molecules	FormField (label+input+error), SearchBar, ProductCard, StatusBadge
Organisms	RegistrationForm, ApplicationWizardStep, OfferSummary, PaymentTimeline

Level	Components
Templates	AuthTemplate, FormTemplate, DashboardTemplate
Pages	LoginPage, ProductListPage, ApplicationFormPage

Theming

- `MaterialTheme` with custom `ColorScheme` (light + dark).
- Primary: brand blue. Secondary: teal. Error: red. Surface: white/dark gray.
- Typography scale: Display (hero numbers), Headline (section titles), Title (card titles), Body (content), Label (form labels, buttons).
- Spacing scale: 4dp base — 4, 8, 12, 16, 24, 32, 48.
- Border radius: 4dp (buttons), 8dp (cards), 12dp (modals), 16dp (bottom sheets).

Component Standards

- All components MUST have `@Preview` (light + dark).
- All interactive components MUST have `contentDescription`.
- All buttons MUST have minimum 48dp touch target.
- All text fields MUST show validation state: default, focused, error, disabled.
- All loading states MUST use Skeleton composable (not spinner).
- All error states MUST show message + retry action.

Icons

- Material Icons (filled variant).
- Custom icons only when Material doesn't cover the use case.
- Icon size: 24dp standard, 20dp in dense layouts, 48dp for primary actions.

Animation

- Compose animation APIs only: `animateContentSize`, `AnimatedVisibility`, `animateFloatAsState`.
- Duration: 200ms (micro-interactions), 300ms (transitions), 500ms (page transitions).
- Easing: `FastOutSlowIn` for enter, `FastOutLinearIn` for exit.

MANDATORY RULES:

- All components MUST follow atomic design hierarchy.
- All components MUST support light + dark theme.
- All interactive elements MUST meet 48dp minimum touch target.
- All components MUST have Compose `@Preview`.

EA12: Compliance Architecture

FCA Consumer Duty

- All fee information MUST be displayed transparently (no hidden fees).
- Plain-language terms required alongside legal text.

- Customers MUST be able to understand total cost before accepting.
- 14-day cooling-off period MUST be communicated.
- Early repayment rights MUST be prominently displayed.

- **Lawful basis:** Contract performance (loan application), legitimate interest (fraud prevention), consent (marketing).
- **Data minimization:** Collect only fields required for loan decision.
- **Right to access:** Customer can download all their data.
- **Right to erasure:** Hard delete with audit trail (exception to soft-delete rule).
- **Right to portability:** Export in machine-readable format.
- **Consent management:** Granular consent per purpose (application processing, marketing, analytics). Marketing consent OFF by default.

PSD2 SCA

- Strong Customer Authentication required for: loan acceptance, payment initiation, viewing sensitive financial data.
- Two factors from: knowledge (password), possession (device/OTP), inherence (biometric).
- Exemptions: viewing product catalog (public data), viewing own application status (session-authenticated).

PRA Operational Resilience

- Important Business Services: loan application processing, loan disbursement, payment collection.
- Impact tolerances defined per service.
- Third-party dependency mapping required.
- Regular scenario testing (e.g., what if KYC provider is down for 4 hours?).

DORA (Digital Operational Resilience Act)

- ICT risk management framework.
- Incident classification and reporting (major incidents within 4 hours).
- Third-party ICT provider register.
- Regular resilience testing.
- Information sharing on cyber threats.

Data Retention

Data	Retention Period
Application data	7 years after loan closure
Rejected applications	3 years
KYC documents	5 years after relationship end
Audit logs	7 years
Marketing consent records	Duration of consent + 1 year

Audit Trail Architecture

- **Storage**: dedicated `audit` schema with append-only tables. No UPDATE or DELETE operations permitted on audit tables (enforced via database permissions).
- **Audit event schema**:

```
{
  "auditId": "uuid",
  "timestamp": "ISO8601",
  "userId": "uuid (or 'system')",
  "action": "CREATE | READ | UPDATE | DELETE | LOGIN | LOGOUT | EXPORT | CONSENT_CHANGE",
  "resource": "entity type (e.g., LoanApplication, User, Offer)",
  "resourceId": "uuid",
  "changes": {"field": {"old": "value", "new": "value"}},
  "reason": "why this action was performed (mandatory for sensitive operations)",
  "ipAddress": "masked (last octet zeroed for GDPR)",
  "correlationId": "uuid",
  "outcome": "success | failure"
}
```

- **What must be audited**: all data access (read/write) to PII fields, all state transitions on loan applications, all authentication events (login, logout, failed attempts, lockout), all consent changes (granted, withdrawn), all data exports (DSAR responses, statements), all administrative actions (user role changes, config changes).
- **Retention**: 7 years (aligned with CCA record-keeping requirements). Archived to cold storage after 2 years. Queryable via audit search API for compliance team.
- **Separation**: audit data stored in separate database (not the application database) to prevent accidental deletion and ensure independence.
- **Query access**: read-only API for compliance and operations teams. No direct database access. All audit queries themselves are audit-logged (who searched what).
- **Performance**: audit writes are asynchronous (via message queue) to avoid impacting application latency. Eventual consistency acceptable (audit record appears within 5 seconds of action).

MANDATORY RULES:

- All fee displays MUST show total cost including ALL fees.
- All PII processing MUST have documented lawful basis.
- All sensitive actions MUST require SCA (two-factor).
- All important business services MUST have defined impact tolerances.
- All data MUST have defined retention period and automated cleanup.

EA13: Code Standards

Backend (C#) Naming Conventions

Element	Convention	Example
Classes	PascalCase	`LoanApplicationService`
Interfaces	I-prefix PascalCase	`ILoanApplicationRepository`
Methods	PascalCase	`SubmitApplication()`
Properties	PascalCase	`ApplicationStatus`
Private fields	_camelCase	`_applicationRepository`
Constants	PascalCase	`MaxRetryCount`
Enums	PascalCase (singular)	`ApplicationStatus.Submitted`

Element	Convention	Example
Async methods	Suffix with Async	`SubmitApplicationAsync()`

Mobile (Kotlin) Naming Conventions

Element	Convention	Example
Classes	PascalCase	`ApplicationFormViewModel`
Functions	camelCase	`submitApplication()`
Properties	camelCase	`applicationStatus`
Constants	SCREAMING_SNAKE	`MAX_RETRY_COUNT`
Composables	PascalCase	`ApplicationFormScreen()`
State	camelCase with Flow suffix	`applicationStateFlow`
Room entities	PascalCase + Entity suffix	`ApplicationDraftEntity`
Room DAOs	PascalCase + Dao suffix	`ApplicationDraftDao`

Code Documentation

- All public APIs: XML doc comments (C#) / KDoc (Kotlin).
- All complex business logic: inline comments explaining WHY (not what).
- All DTOs: property-level documentation for non-obvious fields.
- All configuration: comments explaining purpose and valid values.
- README.md in each project root: setup instructions, architecture overview, key decisions.

Code Quality

- SonarQube quality gate: 0 critical issues, 0 high issues, < 5 medium issues.
- Code coverage: minimum 80% line coverage for Application layer, 60% for Infrastructure layer.
- Cyclomatic complexity: max 10 per method.
- Method length: max 30 lines (excluding boilerplate).
- Class length: max 300 lines.
- No TODO/FIXME in main branch (must be tracked as Jira tickets).

MANDATORY RULES:

- All public APIs MUST have documentation comments.
- All code MUST pass SonarQube quality gate before merge.
- All async methods MUST use Async suffix (C#).
- No TODO/FIXME in main branch.

EA14: Testing Standards

Test Pyramid

Level	Scope	Framework	Target Coverage	Speed
Unit	Single class/method	xUnit + NSubstitute (C#), JUnit + MockK (Kotlin)	80% Application layer	< 1s per test
Integration	Service + DB/cache	Testcontainers + WireMock	All API endpoints	< 5s per test

Level	Scope	Framework	Target Coverage	Speed
Contract	API schema compliance	OpenAPI validator	All endpoints match spec	< 1s per test
E2E (Mobile)	Full user flow	Espresso + Compose UI Test	Critical paths	< 30s per test
Security	OWASP vulnerabilities	OWASP ZAP	All endpoints	Nightly
Performance	Load/stress	k6	p95 targets met under load	Weekly

What to Test at Each Level

Unit tests:

- Domain entities: business rules, state transitions, validation.
- Application handlers: use case logic, edge cases, error paths.
- Validators: all validation rules, boundary values.
- Value objects: equality, formatting, conversion.

Integration tests:

- API endpoints: happy path, validation errors, auth failures, not found.
- Database: migrations run cleanly, queries return correct data, concurrency handling.
- External services: circuit breaker triggers, retry behavior, fallback activation.

Mobile UI tests:

- Form validation: required fields, format validation, error display.
- Navigation: deep links, back stack, screen transitions.
- Offline: form save, sync on reconnect, error display.
- Accessibility: TalkBack navigation, focus order, touch targets.

Test Data Management

- Test fixtures: factory pattern for creating test entities (`ApplicationFactory.CreateSubmitted()`).
- No production data in tests. All test data synthetic.
- Database: Testcontainers spins up fresh SQL Server per test class.
- External services: WireMock stubs for all third-party APIs.
- Sensitive data: use obviously fake data (`John Doe`, `07700900000`, `AA000000A`).

Test Naming Convention

- C#: `MethodName_Scenario_ExpectedResult` (e.g., `SubmitApplication_WhenKycNotComplete_ReturnsValidationError`).
- Kotlin: backtick method names (e.g., `submit application when kyc not complete returns validation error`).

MANDATORY RULES:

- All Application layer handlers MUST have unit tests (80% coverage).

- All API endpoints MUST have integration tests.
- All critical user flows MUST have E2E tests.
- All tests MUST use synthetic data (no production data).
- All PRs MUST pass all tests before merge.

EA15: Feature Flag Standards

Platform

- Azure App Configuration with Feature Management.
- .NET: `Microsoft.FeatureManagement` library.
- Mobile: feature flags fetched on app launch, cached locally (1hr TTL).

Flag Naming Convention

`{context}.{feature}.{variant}` — e.g., `loans.earlyRepayment.enabled`, `kyc.livenessCheck.v2`, `ui.darkMode.enabled`.

Flag Types

Type	Use Case	Example
Release toggle	Gradual rollout of new feature	`loans.offerComparison.enabled`
Ops toggle	Kill switch for degraded dependency	`kyc.provider.enabled`
Experiment toggle	A/B testing	`ui.offerScreen.variant`
Permission toggle	Feature access by role/tier	`loans.premiumProducts.enabled`

Lifecycle

1. Flag created in Azure App Configuration with default OFF.
2. Code deployed behind flag (both paths tested).
3. Flag enabled: internal users -> staging -> 1% prod -> 10% -> 50% -> 100%.
4. After 100% stable for 2 sprints: remove flag, clean up code.
5. Max flag lifetime: 90 days (except ops toggles).

MANDATORY RULES:

- All new features MUST be behind a feature flag.
- All feature flags MUST have an owner and expiry date.
- Both flag-on and flag-off paths MUST be tested.
- Ops toggles MUST be documented in runbooks.

EA16: Dependency Management

Adding New Dependencies

1. Check approved tech stack (EA1). If not listed -> raise `UNLISTED_TECHNOLOGY_REQUIRED`.
2. Verify license compatibility (MIT, Apache 2.0, BSD allowed. GPL, AGPL prohibited).
3. Check Snyk for known vulnerabilities.
4. Check maintenance status (last commit < 6 months, active maintainers).
5. Add to project with pinned version (no floating versions).
6. Document in PR description: why needed, alternatives considered, license.

Updating Dependencies

- Automated weekly PRs via Renovate.
- Patch versions: auto-merge if tests pass.
- Minor versions: manual review, merge within 1 sprint.
- Major versions: ARB review, dedicated upgrade story.

Prohibited Licenses

GPL (any version), AGPL, SSPL, Commons Clause. Any license requiring source code disclosure of proprietary code.

MANDATORY RULES:

- All dependencies MUST have compatible licenses (MIT, Apache 2.0, BSD).
- All dependency versions MUST be pinned (no wildcards).
- All dependency updates MUST pass vulnerability scan.
- GPL/AGPL licensed dependencies are PROHIBITED.

EA17: Documentation Standards

Required Documentation

Document	Owner	Location	Updated
Architecture Decision Records (ADRs)	Tech Lead	`/docs/adr/` in repo	On each decision
API documentation	Backend dev	Auto-generated from OpenAPI spec	On API change
Database schema docs	Backend dev	Auto-generated from EF Core model	On migration
Runbooks	DevOps	Confluence	On infra change
Onboarding guide	Tech Lead	`/docs/onboarding.md`	Quarterly
Incident postmortems	On-call engineer	Confluence	Within 48hr of incident

Architecture Decision Records (ADRs)

Format:

```
# ADR-{NNN}: {Title}

## Status: Proposed | Accepted | Deprecated | Superseded

## Context
What is the issue that we're seeing that is motivating this decision?

## Decision
What is the change that we're proposing and/or doing?

## Consequences
What becomes easier or more difficult to do because of this change?
```

- ADR required for: technology choices, architecture pattern changes, security decisions, third-party integrations.
- ADRs are immutable once accepted. Supersede with new ADR if decision changes.

MANDATORY RULES:

- All architecture decisions MUST have an ADR.
- All APIs MUST have auto-generated documentation from OpenAPI spec.
- All incidents MUST have postmortem within 48 hours.
- All runbooks MUST be reviewed quarterly.

EA18: Notification Architecture

Standards for customer and system notifications across all channels.

Notification Channels

Channel	Technology	Use Case	Delivery SLA
Push notification	Firebase Cloud Messaging (FCM) for Android	Real-time alerts: status changes, payment reminders, security alerts	< 30 seconds
Email	SendGrid or Azure Communication Services	Transactional: confirmations, statements, regulatory notices	< 5 minutes
SMS	Twilio or equivalent gateway	OTP delivery, critical alerts when push unavailable	< 10 seconds
In-app	Local notification store + badge count	Non-urgent: marketing, product updates, tips	On next app open

Notification Categories

Category	Opt-out Allowed	Examples
Security	No	OTP, login from new device, password change, account lockout
Regulatory	No	Arrears notices (CCA s.86B), rate change notices, annual statements

Category	Opt-out Allowed	Examples
Transactional	No	Application status, payment confirmation, disbursement confirmation
Service	Yes (per channel)	Payment reminders, offer expiry warnings, document ready
Marketing	Yes (default OFF, requires GDPR consent)	New products, promotional rates, referral offers

Notification Event Schema

```
{
  "notificationId": "uuid",
  "userId": "uuid",
  "category": "security | regulatory | transactional | service | marketing",
  "channel": "push | email | sms | in_app",
  "template": "template-name-v1",
  "data": {"key": "value — template variables"},
  "priority": "high | normal | low",
  "scheduledAt": "ISO8601 or null for immediate",
  "expiresAt": "ISO8601 or null",
  "deepLink": "/route/to/relevant/screen",
  "correlationId": "uuid"
}
```

Notification Preferences

- Customer can manage preferences per category and per channel (except Security and Regulatory — always on).
- Preferences stored in database, cached in Redis (15min TTL).
- Marketing requires explicit GDPR consent — separate from notification preferences.
- Preference changes take effect immediately (cache invalidated on write).

Template Management

- All notification content managed via templates (not hardcoded).
- Templates versioned: `{name}-v{N}` (e.g., `payment-reminder-v2`).
- Templates support variables: `{{customerName}}`, `{{amount}}`, `{{dueDate}}`.
- Templates reviewed by compliance before activation (regulatory notifications).
- Multi-language support: templates stored per locale (e.g., `en-GB`, `ar-SA`).

Delivery & Retry

- Push: fire-and-forget via FCM. If device token invalid, mark for re-registration.
- Email: async via message queue. Retry 3 times with exponential backoff. Track delivery status (sent, delivered, bounced, opened).
- SMS: async via message queue. Retry 2 times. Track delivery status.
- Failed delivery: log to notification audit table. Do not retry indefinitely.

Deep Linking

- Every notification includes a deep link to the relevant screen.
- Deep link format: app scheme URI (e.g., `loanapp://dashboard/payments`).
- If app not installed (email/SMS): link to app store or web fallback.
- Deep link must work from killed state (Navigation Compose deep links).

MANDATORY RULES:

- Security and regulatory notifications MUST NOT be opt-out-able.
- Marketing notifications MUST be OFF by default and require explicit consent.
- All notification content MUST use templates (no hardcoded text).
- All notifications MUST include a deep link.
- All notification deliveries MUST be audit-logged.
- PII MUST NOT appear in push notification titles (visible on lock screen).

EA19: Payment Card Security (PCI-DSS)

Standards for handling payment card data (debit/credit cards).

PCI-DSS Compliance Approach

- **Strategy: Tokenisation via third-party payment processor.** The application NEVER stores, processes, or transmits raw card numbers (PAN). All card operations are delegated to a PCI-DSS Level 1 certified payment processor.
- **SAQ Level: SAQ A** — all card data functions outsourced to certified third party. No card data touches our servers.

Card Data Handling Rules

Data Element	Can We Store?	Can We Display?	Notes
Full PAN (card number)	NEVER	NEVER	Only payment processor handles
Masked PAN (last 4 digits)	Yes	Yes	For display: `**** **** **** 1234`
Cardholder name	Yes (encrypted)	Yes	Needed for CoP matching
Expiry date	NEVER	NEVER	Only payment processor handles
CVV/CVC	NEVER	NEVER	Must never be stored anywhere
Token (from processor)	Yes	No	Used for recurring payments

Implementation Pattern

1. **Card capture**: use payment processor's hosted fields / SDK (e.g., Stripe Elements, Adyen Drop-in). Card data entered directly into processor's iframe — never touches our frontend or backend.
2. **Tokenisation**: processor returns a token representing the card. Token stored in our database.
3. **Recurring payments**: use stored token to initiate payments via processor API. No card data in our systems.
4. **Card updates**: if card expires or is replaced, processor handles via Account Updater service.

Payment Processor Integration

- All communication via TLS 1.2+ encrypted API.

- API keys stored in Azure Key Vault (never in code).
- Webhook for payment status updates (signed with HMAC for verification).
- Idempotency key on all payment requests.

3D Secure (SCA for Card Payments)

- All card payments must support 3D Secure 2.0 (3DS2) for Strong Customer Authentication.
- Payment processor handles 3DS2 challenge flow.
- Exemptions: recurring payments after initial SCA, low-value transactions (per PSD2 RTS).

MANDATORY RULES:

- Raw card data (PAN, expiry, CVV) MUST NEVER touch our systems.
- Card capture MUST use payment processor's hosted fields/SDK.
- All card tokens MUST be stored encrypted.
- All card payments MUST support 3D Secure 2.0.
- PCI-DSS SAQ A compliance MUST be maintained and audited annually.

EA20: Localisation & Multi-Platform

Standards for language support and platform strategy.

Localisation (i18n/l10n)

- **Default locale**: `en-GB` (English, United Kingdom).
- **Architecture**: all user-facing strings externalised into resource files — no hardcoded text in code.
- **Android**: `res/values/strings.xml` (default), `res/values-ar/strings.xml` (Arabic), etc.
- **Backend**: error messages and notification templates stored with locale key.
- **Date/time**: always stored as UTC in database. Displayed in user's timezone. Format per locale (e.g., `dd/MM/yyyy` for en-GB, `yyyy/MM/dd` for ar-SA).
- **Currency**: always stored as minor units (pence/halalas) in database. Displayed with locale-appropriate formatting (e.g., `GBP 1,234.56`, `SAR 1,234.56`).
- **RTL support**: if Arabic is supported, all layouts must support right-to-left (RTL). Jetpack Compose handles RTL automatically if `LayoutDirection` is set correctly.
- **Number formatting**: locale-aware (e.g., decimal separator: `.` for en-GB, may differ for other locales).

Adding a New Locale

1. Create resource files for the new locale.
2. Translate all strings (professional translation, not machine).
3. Test all screens in new locale (layout, truncation, RTL if applicable).
4. Add locale to notification template store.
5. Add locale to backend error message catalogue.

Multi-Platform Strategy

- **Primary platform**: Android (Kotlin/Jetpack Compose) — as defined in EA1.

- **Future platforms**: if iOS or web is added, use platform-native development (Swift/SwiftUI for iOS, React/Next.js for web). No cross-platform frameworks (Flutter, React Native) — per ARB decision.
- **API-first**: backend APIs are platform-agnostic. Same API serves Android, iOS, and web.
- **Feature parity**: all platforms must implement the same features. Platform-specific features (e.g., BiometricPrompt on Android, Face ID on iOS) are abstracted behind a common interface.

App Store & Release Management

- **Android**: Google Play Console. Internal testing track -> Closed beta -> Open beta -> Production (staged rollout: 1% -> 10% -> 50% -> 100%).
- **iOS (future)**: App Store Connect. TestFlight for beta. Phased release (1% -> 100% over 7 days).
- **Release cadence**: every 2 weeks (aligned with sprint). Hotfixes can be released ad-hoc.
- **Version numbering**: `{major}.{minor}.{patch}` (e.g., `1.2.3`). Major = breaking changes, Minor = new features, Patch = bug fixes.
- **Force update**: if a critical security fix is deployed, the app can force users to update via a minimum version check on app launch.
- **Soft update**: for non-critical updates, show a dismissible banner suggesting update.

MANDATORY RULES:

- All user-facing strings MUST be externalised (no hardcoded text).
- All dates/times MUST be stored as UTC and displayed in user's timezone.
- All currency MUST be stored as minor units (pence/halalas).
- If RTL locale is supported, all layouts MUST be tested in RTL mode.
- All releases MUST go through staged rollout (never 100% on day 1).
- Force update mechanism MUST be implemented for critical security fixes.